Was ist DisasterDAO?

Gemeinsame Soforthilfe mit festen Regeln

Dieser Bericht erklärt die Idee von Grund auf, dokumentiert die bereits umgesetzte Anwendung und fasst das abgeschlossene technische Review zusammen. Anschließend zeigt er, welche Schritte vor der Hackathon-Abgabe und für eine mögliche Weiterentwicklung sinnvoll wären.

Das Problem, das die Idee untersuchen soll

Hilfsorganisationen könnten Geld gemeinsam für Katastrophen zurücklegen. Im Ernstfall müssen sie entscheiden: Wer erhält wie viel, aufgrund welcher Informationen und mit wessen Zustimmung? DisasterDAO untersucht, ob vorab vereinbarte und öffentlich überprüfbare Regeln diese Freigabe unterstützen können. Dass reale Organisationen dieses System benötigen oder bevorzugen, wurde noch nicht durch Interviews belegt.

Die Lösung in einem Satz

Ein gemeinsamer digitaler Fonds zahlt einen festen Betrag an vorher bestimmte Organisationen aus, wenn überprüfbare Ereignisdaten die vereinbarten Bedingungen erfüllen; alternativ können zwei von drei Verantwortlichen die Zahlung genehmigen.

Was heute tatsächlich existiert

Ein funktionsfähiger Hackathon-Prototyp: Drei fiktive Organisationen haben jeweils einen einmaligen Anspruch auf 1.000 wertlose Demotoken. Ein echter, digital signierter Datensatz zum Türkei-Erdbeben vom 6. Februar 2023 kann eine Auszahlung auslösen. Ein gemeinsamer Notfallweg und eine einfache deutschsprachige Oberfläche sind ebenfalls umgesetzt.

Nachgewiesen: Eine reguläre Auszahlung und eine Notfallauszahlung wurden auf einer öffentlichen Test-Blockchain bestätigt. Für die dritte Organisation ist noch ein Anspruch offen. Das Review hat konkrete Fehler in Hilfsskripten und Darstellung gefunden und behoben. Es wurde kein kritischer Contract-Fehler festgestellt; dies ersetzt kein unabhängiges Sicherheitsaudit.

Leseweg

Seite 2: Begriffe und Beispielablauf. Seite 3: Regeln des Fonds. Seite 4: Umsetzung und Bedienung. Seite 5: öffentliche Nachweise. Seite 6: Review und Korrekturen. Seite 7: verbleibende Grenzen. Seite 8: priorisierte Zukunftsschritte. Seite 9: Übergabe und Quellen.

So funktioniert der Ablauf

Die wichtigsten Begriffe

NGO: Eine Hilfsorganisation. Die drei NGOs in diesem Projekt sind ausdrücklich fiktive Demo-Empfänger.

Blockchain: Ein gemeinsam geführtes Transaktionsregister. Für diesen Prototyp wird Base Sepolia verwendet, eine Testumgebung ohne produktive Auszahlungsmittel.

Smart Contract: Ein Programm auf der Blockchain, das Regeln und Zahlungen ausführt. Es startet nicht von selbst: Jemand muss eine Transaktion einreichen und deren Gebühren bezahlen.

Wallet: Ein Konto mit einer öffentlichen Adresse und einem privaten Schlüssel. Der Schlüssel berechtigt zur Unterzeichnung von Aktionen und bleibt ausschließlich lokal.

Token: Eine digital verbuchte Einheit. dUSD ist hier ein selbst erstellter Demotoken ohne Wert. Test-ETH bezahlt Blockchain-Gebühren; aUSD bezahlt API-Abfragen. Diese drei Einheiten haben verschiedene Aufgaben.

API und Airnode: Eine API stellt Daten für Programme bereit. Hier stammen die Erdbebendaten von USGS. Ein API3-Airnode ruft sie ab und signiert seine Antwort digital. USGS selbst betreibt diesen Signierer nicht.

Quorum: Eine vorgeschriebene Mindestzahl von Zustimmungen. Hier müssen zwei unterschiedliche Konten eines dreiköpfigen Notfallgremiums zustimmen.

Beispiel: reguläre Soforthilfe

1. Der Fonds erhält 3.000 dUSD und wird aktiviert. Empfänger und Freigaberegeln stehen bereits fest.
2. Ein Nutzer ruft über den Airnode die Daten des festgelegten historischen Erdbebens ab. Der Airnode liefert die Daten mit Signatur und Signaturzeit.
3. Der Nutzer reicht die ursprüngliche Antwort beim Contract ein.
4. Der Contract prüft Herkunft, Anfrage, Alter und den Inhalt der Antwort.
5. Bei erfüllten Regeln erhält die ausgewählte NGO 1.000 dUSD. Der Einreicher kann die Zahlung nicht auf seine eigene Adresse umlenken.
6. Der Anspruch dieser NGO ist verbraucht und die Zahlung erscheint in der Historie.

Wenn Daten fehlen

Zwei Gremiumsmitglieder können dieselbe NGO und dieselbe Begründung genehmigen. Danach kann die Zahlung ausgeführt werden. Auch dieser Weg verbraucht den einzigen Anspruch der NGO. Er schafft kein zusätzliches Budget.

Welche Regeln wurden umgesetzt?

Ein bewusst kleiner, unveränderlicher Einsatz

Der MVP behandelt genau ein historisches Ereignis: USGS **us6000jllz**, das Türkei-Erdbeben vom 06.02.2023. Die historische Beobachtung wird heute erneut abgefragt. Deshalb sind Beobachtungszeit und Zeitpunkt der digitalen Signatur getrennte Angaben.

Empfänger und Budget: Drei feste NGO-Adressen; je 1.000 dUSD, insgesamt höchstens 3.000 dUSD. Der Demotoken besitzt sechs Nachkommastellen und eine feste Gesamtmenge von 3.000 dUSD.

Aktivierung: Erst bei vollständiger Finanzierung. Jeder darf den vollständig finanzierten Fonds aktivieren; dadurch erhält niemand Änderungs- oder Abzugsrechte.

Authentifizierung: Der Contract akzeptiert nur den festgelegten Airnode und die festgelegte Anfrage. Er prüft die Signatur über die unveränderten Antwortbytes. Vom Einreicher beigefügte Vertrauensangaben ersetzen diese Vorgaben nicht.

Frische: Die Signatur darf höchstens 300 Sekunden alt sein, nicht in der Zukunft liegen und nicht vor der Fondsaktivierung entstanden sein.

Ereignisbedingungen: Ereignis-ID passend, Typ „earthquake“, Status „reviewed“, Magnitude mindestens 7,0; Beobachtung am 06.02.2023 nach UTC; Gebiet 36 bis 39 Grad Nord und 35 bis 39 Grad Ost.

Fehlende Werte: Fehlende oder null gesetzte erforderliche Daten werden abgelehnt. Sie gelten nicht als gültiger Messwert unterhalb einer Schwelle.

Schutz vor Doppelzahlung: Beide Freigabewege prüfen denselben Auszahlungsstatus pro NGO. Weder ein neuer Datensatz noch eine weitere Begründung erzeugt einen zweiten Anspruch.

Notfallgremium: Drei festgelegte, unterschiedliche Konten, getrennt von allen NGO-Adressen. Zwei verschiedene Mitglieder müssen genau derselben Kombination aus Empfänger und Begründungshash zustimmen.

Technische Absicherung

Der Contract aktualisiert den Anspruch vor dem Token-Transfer und verwendet sichere Transferfunktionen sowie einen Schutz gegen verschachtelte Wiedereintritte während einer Zahlung. Schlägt ein Transfer fehl, werden Anspruch und Budget nicht verbraucht.

Es gibt keine administrativen Regeländerungen, Upgrades oder beliebigen Abzüge. Änderungen dieser Regeln würden einen neuen Contract erfordern. Eine Rückzahlung überschüssig eingezahlter Token ist ebenfalls nicht vorgesehen.

Was wurde programmiert und vorbereitet?

1. Contract und Datenanbindung

DisasterFund.sol verwaltet Finanzierung, Aktivierung, Ansprüche und beide Auszahlungspfade. **DemoUSD.sol** stellt das feste Demo-Vermögen bereit. **ResponseProbe.sol** diente dazu, echte USGS-Daten zunächst ohne Fondsrisiko in Solidity zu prüfen. Die Verifikation folgt dem offiziellen API3-Starter und nutzt bewährte OpenZeppelin-Bausteine.

Die Airnode-Anfrage projiziert Ereignis-ID, Magnitude, Beobachtungszeit, Koordinaten, Status und Ereignistyp. Echte signierte Antworten sind als Fixtures gesichert. Eine zweite echte Anfrage enthält zusätzlich die Tiefe und dient dem Negativtest „andere Anfrage“.

2. Oberfläche

Das deutschsprachige Portal zeigt Poolstand, feste Regeln, Empfänger, offene Ansprüche, Gremiumsfreigaben und bestätigte Zahlungen. Eine Browserwallet kann Transaktionen signieren. Im Jurybereich lassen sich originale Antwortdateien laden, ohne Zahlung simulieren oder zur Auszahlung einreichen. Vorschauwerte werden ausdrücklich als unverifiziert gekennzeichnet.

3. Skripte und reproduzierbare Werkzeuge

Vorhanden sind Skripte für bezahlten API-Abruf, Anfragehash, Signaturprüfung, Rollenanlage, Gebührenschätzung, Deployment, Deploymentprüfung, Einreichung, Manipulationstests, Notfallzahlung und Evidenzbericht. Ein lokales Testnetz ermöglicht wiederholbare Abläufe mit echten Signaturen und einer ausdrücklich historischen Testuhr.

Technik: Solidity und Foundry für Contracts; TypeScript, viem und x402 für die Daten- und Transaktionsskripte; React und Vite für die Oberfläche; Playwright für Browsertests. Die Abhängigkeiten und übernommenen Lizenzen sind im Repository dokumentiert.

4. Demo und Abgabeunterlagen

Ein 74,6 Sekunden langes Video mit deutschen Texteinblendungen zeigt den vollständigen lokalen Ablauf und anschließend die realen Testnet-Nachweise. Es besitzt keinen Ton. README, Demo-Anleitung und ein Devfolio-Entwurf erklären Einrichtung, Nutzung, Nachweise und Grenzen. Ein Video-Upload und die eigentliche Einreichung wurden nicht durchgeführt.

5. Umgang mit Schlüsseln und Änderungen

Projekt- und Rollenschlüssel bleiben in ignorierten lokalen Umgebungsdateien. Der Browser erhält keine privaten Schlüssel. Geprüfte Meilensteine wurden mit klaren Commit-Nachrichten auf den bestehenden Branch origin/main gepusht. Das originale Übergabedokument wurde nicht verändert.

Welche Ergebnisse sind öffentlich nachweisbar?

Netzwerk: **Base Sepolia**, Chain-ID 84532. Die folgenden Links führen zum öffentlichen Blockchain-Explorer und ermöglichen die unabhängige Betrachtung der Transaktionen.

Die deployten Programme

Fonds: 0x8b1cad09c9be9d2e1ff38f4a490951bc1a317257

Demotoken: 0xdc6ab1e15c3f23b05276fde1ac4a7311abe8e914

Bereits bestätigte Zahlungen

NGO 1: 1.000 dUSD über den regulären Datenpfad. Oracle-Auszahlung ansehen

NGO 2: 1.000 dUSD über zwei Gremiumsfreigaben. Notfall-Auszahlung ansehen

NGO 3: Der dritte Anspruch wurde für weitere Jurytests offengelassen. Beim letzten Abgleich waren insgesamt 2.000 dUSD ausgezahlt. Damit verbleiben 1.000 dUSD im Fonds; vor späteren Tests den Status erneut prüfen.

Die vorgeschriebenen Ablehnungsfälle

Am deployten Contract wurden vier Fälle erfolgreich abgewiesen: ein nachträglich veränderter Messwert, eine echte zu alte Antwort, eine echte Antwort auf eine andere Anfrage sowie eine fremde Signatur. Diese Ablehnungen wurden durch lesende Contract-Simulationen geprüft. Es wurden dafür keine absichtlich scheiternden öffentlichen Transaktionen gesendet.

Kosten und aktueller Abgleich

Die vollständige Einrichtung einschließlich Finanzierung und Aktivierung kostete rund **0,0000202034 Test-ETH**: 0,0000198084 für Ausführung und 0,0000003950 für zusätzliche Datengebühren. Beim Review betrug das Guthaben der Projektwallet rund 0,0001697766 Test-ETH. Gebühren und Guthaben sind Momentaufnahmen, keine Zusage für spätere Ausführungen.

Zusätzlich wurden die vollständigen laufenden Bytecodes beider deployten Programme mit lokal simulierten Erstellungen aus dem aktuellen Build verglichen. Sie stimmen einschließlich der im Constructor festgelegten Werte überein. Signierer, Anfrage, Token, Empfänger, Gremium und Aktivierung wurden ebenfalls abgeglichen.

Review: Fehler und durchgeführte Korrekturen

Geprüft wurden Contract-Regeln, Signatur- und Zeitprüfung, Budget und Quorum, Skripte, Oberfläche, Dokumentation, Abhängigkeiten und der laufende Testnet-Code. Das Ergebnis ist ein technisches Eigenreview mit gezielten Tests; eine vollständige unabhängige Sicherheitsprüfung fand nicht statt.

Gefunden und behoben

API-Abruf konnte unnötig bezahlen: Wenn die gewünschte Ausgabedatei bereits existierte, konnte der bezahlte Abruf vor dem Schreibfehler stattfinden. Die Datei wird jetzt vor bezahlter Arbeit exklusiv reserviert. Tests prüfen bestehende Dateien, parallele Aufrufe und einen erneuten Versuch nach API-Fehlern.

Historie konnte Zahlungen unterschlagen: Mehrere NGO-Zahlungen in derselben Transaktion wurden nach Transaktionshash zusammengefasst. Die Historie unterscheidet jetzt einzelne Ereignisse anhand von Hash und Log-Index. Ein Regressionstest prüft mehrere Zahlungen und doppelt gelieferte Daten.

Quorumanzeige konnte veraltet sein: Nach einem Wechsel der Begründung konnten alte Freigaben noch erscheinen; verspätete Antworten konnten neuere Anzeigen überschreiben. Freigaben sind jetzt an Empfänger und Begründung gebunden, veraltete Antworten werden verworfen. Ein Browsertest verzögert gezielt eine RPC-Antwort.

Begründung war missverständlich beschrieben: Die Oberfläche sprach von einer öffentlichen Begründung, obwohl nur deren Hash gespeichert wird. Sie erklärt nun, dass der genaue Text separat geteilt und aufbewahrt werden muss und äußere Leerzeichen entfernt werden.

Zusätzliche Absicherung

Ergänzt wurden Contract-Tests für geänderte Signaturzeit, fehlerhafte und verformte Signaturen, alle drei möglichen Zweierkombinationen des Gremiums sowie einen Token-Callback, der während einer Zahlung einen weiteren Anspruch auszuführen versucht. Auch der Bytecode-Abgleich ist jetzt Teil der Deploymentprüfung.

Abschließende Prüfergebnisse

20 TypeScript-Tests, 33 Solidity-Tests und 5 Browsertests bestanden. Die Solidity-Suite enthält 1.000 Fuzz-Durchläufe für die Mischung beider Auszahlungspfade. Typprüfung und Produktionsbuild bestanden. npm audit meldete null bekannte Schwachstellen. Die tatsächlichen lokalen privaten Projektschlüssel wurden weder in den 599 geprüften getrackten Dateien noch im Produktionsbuild gefunden; die Umgebungsdateien erscheinen nicht in der geprüften Git-Dateihistorie.

Die Build-Ausgabe weist weiterhin auf eine große JavaScript-Datei und ein für den Browser ausgelagertes Node-Modul einer Abhängigkeit hin. In den ausgeführten Browserabläufen trat dadurch kein Fehler auf. Dies sind verbleibende technische Hinweise, keine nachgewiesenen Sicherheitslücken.

Was bleibt offen oder bewusst eingeschränkt?

Wichtige Vertrauensgrenzen

Eine Person kontrolliert die Demorollen: Sechs getrennte Schlüssel simulieren unabhängige Beteiligte. Tatsächlich werden sie im Prototyp von einer Person gehalten. Für reale Mittel wäre das keine unabhängige gemeinsame Kontrolle.

Der Notfallweg ist eine bewusste Ausnahme: Zwei Gremiumsmitglieder können auch ohne nachgewiesenen Oracle-Ausfall freigeben. Zustimmungen haben keine Ablauffrist und können nicht zurückgenommen werden. Die Einmaligkeit und feste Auszahlungshöhe bleiben wirksam.

Signierte Daten sind kein Bedarfsnachweis: Der Fonds vertraut einem einzelnen API3-Signierer und USGS als Quelle. Die Signatur belegt die konkrete Antwort unter diesen Annahmen, aber weder humanitären Bedarf noch korrekte Mittelverwendung. Der historische Einsatz validiert keine produktive Katastrophenhilfe.

Technische und betriebliche Restpunkte

Deployment besteht aus mehreren Schritten: Token, Fonds, Finanzierung und Aktivierung sind getrennte Transaktionen. Die fertige Konfigurationsdatei wird erst am Ende geschrieben. Bei Abbruch können Zwischenstände existieren; ein belastbares Wiederaufnahmeprotokoll fehlt. Vor einem erneuten Versuch müssen bereits gesendete Transaktionen geprüft werden.

Begründung und Nachprüfung: Auf der Chain liegt nur der Hash. Ohne separat erhaltenen Text ist die Begründung nicht lesbar. Die spätere Nachprüfung ist eine dokumentierte Prozesspflicht und wird nicht technisch erzwungen.

Verfügbarkeit der Anzeige: Die Oberfläche hängt am öffentlichen RPC-Dienst. Ein Ausfall oder fehlende Historienquittungen können das Laden beeinträchtigen. Eine umfassende Behandlung von Blockchain-Reorganisationen und getrennte Fehlerzustände pro Anzeige fehlen.

Datenformate und Grenzen: Der JSON-Leser arbeitet mit festgelegter Genauigkeit und unterstützt nicht jedes denkbare Zahlenformat. Koordinaten werden auf sechs Dezimalstellen gekürzt. Der konkrete echte USGS-Datensatz wurde erfolgreich verarbeitet; andere Quellen benötigen eigene Prüfungen.

Unveränderlichkeit hat Folgen: Verlorene NGO-Schlüssel, ausgefallene Gremiumsmitglieder und überschüssige Einzahlungen lassen sich über den bestehenden Contract nicht administrativ korrigieren. Es gibt keine Auszahlungsfrist oder Rückabwicklung.

Das Review hat unter dem beschriebenen Testmodell keinen kritischen Contract-Fehler gefunden. Diese Aussage ist auf die geprüften Pfade und die Demo-Token-Annahmen begrenzt. Formale Verifikation, ein unabhängiges Audit und ein Produktionsbetrieb waren nicht Bestandteil der Arbeit.

Was würde ich als Nächstes umsetzen?

Die folgende Reihenfolge ist eine Empfehlung. Sie stellt keine bereits beauftragte Erweiterung dar. Neue Funktionen sollten den kleinen Hackathon-Nachweis nicht gefährden. Die dokumentierte Deadline ist der 31.10.2026 um 15:00 Uhr MEZ; vor Abgabe ist sie im Veranstalterportal erneut zu prüfen.

Priorität 0: vorhandenen MVP sauber abgeben

1. Repository-Sichtbarkeit prüfen und nach bewusster Freigabe öffentlich machen. Beim zuletzt dokumentierten Stand war es privat.
2. Installation in einer frischen Umgebung nachvollziehen und Jury-Anleitung einmal vollständig durchspielen.
3. Video ansehen, hochladen, Devfolio-Entwurf vervollständigen und tatsächliche Abgabe bestätigen.
4. Für eine neue vollständige Live-Demo einen neuen Fonds bereitstellen und erst danach frische signierte Daten abrufen. Die historische Originalsignatur darf nicht durch Umschreiben ihres Zeitstempels „verjüngt“ werden.

Priorität 1: Verlässlichkeit und Nachvollziehbarkeit verbessern

Deployment wiederaufnehmbar machen: Jeden Transaktionsschritt mit Hash und Status dauerhaft protokollieren, Teilzustände erkennen und vor dem Wiederholen prüfen. Doppelte Deployments und verlorene Zwischenstände vermeiden.

Oberfläche robuster machen: Fondsstatus unabhängig von Historienfehlern laden; klare Fehler- und Ladezustände zeigen; Walletwechsel automatisch anzeigen; Reorganisationen und RPC-Ausfälle behandeln.

Begründung nachvollziehbar aufbewahren: Ein gemeinsam zugängliches, versioniertes Verfahren für den exakten Begründungstext und seine Hash-Zuordnung ergänzen. Personenbezogene Rohbelege gehören nicht in ein öffentliches Register.

Priorität 2: Voraussetzungen eines echten Piloten klären

Mit drei bis fünf NGO-, Fonds- und Einsatzverantwortlichen vergangene Freigabefälle rekonstruieren und mit einer zentralen Lösung vergleichen. Erst mit validiertem Bedarf institutionell getrennte Schlüssel, verlässliche Auszahlungsmittel, Verantwortlichkeiten, Schlüsselausfallverfahren und eine externe Sicherheitsprüfung planen.

Für ein neues Contract-Design wären widerrufbare oder befristete Notfallzustimmungen und Regeln für abgelaufene Einsätze zu prüfen. Solche Änderungen benötigen klare Produktentscheidungen und einen neuen Contract; sie werden nicht nachträglich in das bestehende Deployment eingebaut.

Priorität 3: fachlicher Ausbau nach Validierung

Weitere Einsätze und Datenquellen, zusätzliche begrenzte Tranchen mit eigenem Antrag und Budget, vertrauliche Berichte, nachvollziehbare Prüferbestätigungen und Buchhaltungsexporte. Fortbestehender Bedarf und Mittelprüfung müssen fachlich sauber verbunden werden. Governance-Token, Multichain und KI-Belegbewertung sind derzeit keine notwendige Fortsetzung.

Übergabe, Nachweise und Quellen

Wo die Arbeit liegt

Repository: NoMelatonin / API3_Hackathon_Agent

Projektordner: API3_Project. Einstieg: README.md. Regeln: contracts/DisasterFund.sol und contracts/DemoUSD.sol. Oberfläche: web/main.tsx. Skripte: src/. Tests: test/, src/*.test.ts und e2e/fund.spec.ts. Öffentliche Konfiguration: public/deployment.json. Nachweise: deployments/demo-report.json und deployments/tamper-report.json. Video: demo/DisasterDAO-demo.webm. Abgabeunterlagen: docs/DEMO.md und docs/DEVFOLIO.md.

Nützliche Befehle

npm ci installiert den festgehaltenen Abhängigkeitsstand.
npm run forge -- build kompiliert die Contracts.
npm run dev startet die Oberfläche unter http://127.0.0.1:5173/.
npm test und **npm run forge -- test -vv** prüfen Skripte und Contracts.
npm run build prüft Typen und erzeugt die Produktionsoberfläche.
npm run verify-deployment vergleicht das bestehende Testnet-Deployment lesend mit dem Build.
Browser- und lokale Demotests benötigen die im README beschriebene frische lokale Chain.

Implementierungs- und Review-Meilensteine

8dd4e1a: Werkzeugbasis und USGS-Machbarkeit.
b0ef7dd: echte signierte USGS-Daten in Solidity nachgewiesen.
f6a7dba: unveränderlicher Fonds und gemeinsame Anspruchsgrenze.
ec3fe4d: NGO-Portal und Base-Sepolia-Demo.
43e7154: Demo-Video und Abgabeunterlagen.
54300a4: Dateireservierung vor bezahlten API-Abrufen.
ec9d7ae: vollständige Auszahlungshistorie und Schutz vor veralteter Quorumanzeige.
59cff14: zusätzliche Contract-Angriffstests und vollständiger Bytecode-Abgleich.

Technische Quellen und Prüfgrundlage

API3: Attestation und Signaturformat - im Review direkt abgeglichen.
Offizieller API3-Starter - übernommener Stand 52933ae1fb9371446a927338473466c3d320e19a.
USGS-Airnode - Integration und erlaubter Signierer.
Hackathon/API3-Track - Abgabeanforderungen gemäß vorhandener Übergabe und Projektprüfung.

Grundlage dieses Berichts: Übergabedokument, implementierter Quellcode, vorhandene Deployment- und Demo-Nachweise sowie die in diesem Review erneut ausgeführten Tests, Abhängigkeitsprüfung und Live-Bytecodeverifikation. Frühere Produktideen, implementierter MVP und zukünftige Empfehlungen sind getrennt beschrieben. Der Bericht enthält keine privaten Walletschlüssel.